ASSIGNMENT 3 – CO3

NAME:P.B.VAMSI KRISHNA
192525147

CO3 Coverage: Construct C++ programs using constructors, destructors and operator overloading to control object creation, destruction and operator behaviour – default, parameterised, overloaded, copy and dynamic constructors, destructors, defining and overloading operators, and the rules for operator overloading.

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

Submission Date: 11-11-26

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5

level) Mandatory Execution and GitHub Submission

• Every student must individually execute every task in this assignment, including all "Complete the Code" programs.

• Create a GitHub repository named DSA01-<RegisterNumber>-Assignment3 and share it with the course faculty (public, or private with the faculty added as a collaborator).

• Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C++ source files, input files and output screenshots.

• Include a README.md with your name, register number, compiler and version (for example g++ 13), the exact g++ command used to compile (for example g++ -std=c++17 -Wall -Wextra), the steps to run each program, and a short note of the result of each task.

• Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.

• Submit the GitHub link below and on the cover page of your report. An assignment submitted without a working GitHub link will not be evaluated.

GitHub Repository Link: https://github.com/VAMSI00927/DSA0120--C-.git

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Predict the Output: Write the exact output of the given program before running it. Then run it

and explain every difference between your prediction and the actual output, using the C++ rules involved. Predictions written after running the program will not be credited.
Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Assignment Principle

Every question asks you to design, implement, analyse and evaluate the solution yourself, so that you demonstrate evidence of:

Design → Implementation → Analysis → Evaluation

Q1 – Constructors and Constructor Overloading 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Library Book Records

A library creates Book objects in several situations: a new book with full details, a donated book with only a title, and an empty record that will be filled later. Some records were created with garbage values in the number of copies.

Implementation Tasks

You must:

• Write a default constructor, a parameterised constructor and overloaded constructors for the three situations

• Use a member initializer list in every constructor

• Use constructor delegation to avoid repeating code

• Show that no Book can be created with an invalid number of copies

Design & Analysis

You must:

• Explain when the compiler supplies a default constructor and when it does not

• Explain why member initializer lists are preferred to assignment inside the constructor body

• Explain the order in which members are initialised (declaration order, not list order) •

Draw the state of three objects after construction

Experimental Investigation

You must:

• Create Book objects with every constructor and print them

• Remove the default constructor and record the compiler error when an array of Books is created

• Demonstrate a member initialised in the wrong order and the -Wreorder warning •

Test a negative number of copies

Design Question

When should a constructor be marked explicit? Show a call that compiles without explicit and fails with it.

Alternative Concept and Complexity Analysis

If one constructor with default arguments (Book(std::string t = "", int copies = 0, …)) were used instead of several overloaded constructors and delegation, what would the solution be? Analyse the time and space of object creation (O(1) in both) and compare which design checks the number of copies in fewer places.

Q2 – Copy Constructors: Shallow and Deep Copies 7 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Hospital Prescription Records

A Prescription object holds a dynamically allocated array of medicine names. When a doctor copies a prescription to modify it for a follow-up visit, the original prescription also changes, and the program sometimes crashes when it ends.

Implementation Tasks

You must:

• Implement the class first without a copy constructor and observe the problem

• Implement a deep-copying copy constructor, a copy assignment operator and a destructor

• Verify that copies are independent

• Run the program under -fsanitize=address (or valgrind) for both versions

Design & Analysis

You must:

• Draw memory diagrams for a shallow copy and a deep copy

• Explain why the shallow version crashes at the end (double delete) • State

the Rule of Three and explain why the three functions go together • Explain

the three situations in which the copy constructor is called

Experimental Investigation

You must:

• Add a message in the copy constructor and count how many copies are made when a Prescription is passed by value and by const reference

• Measure the time to copy prescriptions of 10, 1,000 and 100,000 medicines •

Report the sanitizer output for both versions

• Test self-assignment (p = p)

Conclusion

When can you rely on the compiler-generated copy constructor, and when must you write your own?

Complete the Code

Complete TODO 1 to TODO 3 below so that every copy is independent, and execute the program under -fsanitize=address.

```
#include <iostream>

#include <string>


class Prescription {

 int n;

 std::string* medicines; // dynamically allocated array


public:

 explicit Prescription(int count) : n(count), medicines(new std::string[count]) {}
 // TODO 1: Copy constructor: allocate a NEW array of the same size and copy every
```

```
// medicine (deep copy)



 // TODO 2: Copy assignment: guard against self-assignment, build a new array, copy,
// release the old array and return *this



 ~Prescription()
 {
 // TODO 3: Release the dynamically allocated array


 }

 void set(int i, const std::string& m) { medicines[i] = m; }
 std::string get(int i) const { return medicines[i]; }
};

int main()
{
 Prescription p(2);
 p.set(0, "Paracetamol");
 p.set(1, "ORS");
 Prescription q = p; // copy constructor
 q.set(1, "Cetirizine");
 Prescription r(1);
 r = p; // copy assignment
 p.set(0, "Ibuprofen");
 std::cout << p.get(0) << ' ' << p.get(1) << " | " << q.get(0) << ' ' << q.get(1)
<< " | " << r.get(0) << ' ' << r.get(1) << '\n';
```

}

Expected output:

Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS

Delete your copy constructor, run the program again under the sanitizer and record the error reported.

Alternative Concept and Complexity Analysis

If the medicine names were stored in a std::vector<std::string> instead of a dynamically allocated array, the compiler-generated copy constructor would already make a deep copy (the Rule of Zero). What would the solution be? Analyse the time O(n) and space O(n) to copy a prescription of n medicines in both designs, and compare them with a shallow copy (O(1)).

Q3 – Dynamic Constructors and Destructors 7 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Auditorium Seat Booking

An auditorium's seat map is created when an event is set up; the number of rows and seats per row differs from event to event. When the event object is destroyed, all memory must be released and a closing summary written.

Implementation Tasks

You must:

• Write a dynamic constructor that allocates a two-dimensional seat map with new •

Write a destructor that releases every allocation and prints a summary • Create events

on the stack, on the heap (new/delete) and inside a block • Book and cancel seats

Design & Analysis

You must:

• Explain the order in which constructors and destructors run for local, block-scope and dynamically created objects
• Explain what happens if delete is forgotten, and if delete is used instead of delete[] •

Explain why the destructor takes no arguments and cannot be overloaded • Draw the

memory layout of the seat map

Experimental Investigation

You must:

• Print messages in the constructor and destructor and record the order for each kind of object

• Run with -fsanitize=address, first with a missing delete and then with the correct code •

Measure the memory allocated for 10 × 20, 100 × 200 and 1,000 × 2,000 seat maps • Test an

event with zero rows

Conclusion

How does tying memory release to the destructor (the RAII idea) make the program

safer? Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>
#include <string>

struct Logger {
 std::string n;
 Logger(const std::string& s) : n(s) { std::cout << '+' << n << ' '; }
 Logger(const Logger& o) : n(o.n + "c") { std::cout << "copy" << n << ' '; }
~Logger() { std::cout << '-' << n << ' '; }
};

void use(Logger) { std::cout << "use "; }

int main()
 {
 Logger a("A");
```

```
{
Logger b("B");
}
use(a);
Logger* p = new Logger("P");
delete p;
std::cout << "end ";
}
```

My predicted output (write before running):

Explain when each object was created and destroyed, and why the copy was made and destroyed around use().

Alternative Concept and Complexity Analysis

If the seat map were one block of rows × seats elements (index r × seats + s) or a std::vector<std::vector<char>>, instead of an array of row pointers each allocated with new, what would the solution be? Analyse the access time (O(1) in all), the number of allocations (1 versus rows + 1) and the extra space for the row pointers.

Q4 – Overloading Arithmetic and Stream Operators 7 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Money Arithmetic for a Billing Counter

A billing counter adds, subtracts and compares amounts of money stored in rupees and paise. The present code calls functions such as add(a, b) and print(c), which makes the bill calculation hard to read.
Implementation Tasks

You must:

• Write a Money class storing paise in a long long

• Overload +, -, +=, == and < as appropriate

• Overload << (and >>) for printing and reading amounts

• Write the bill calculation using the overloaded operators

Design & Analysis

You must:

• Explain why + returns a new object while += returns a reference to *this • Decide which operators are members and which are friends, and justify each • Explain why << must be a non-member function

• Show how a + b is translated by the compiler into a function call

Experimental Investigation

You must:

• Test addition that carries paise into rupees (Rs 0.75 + Rs 0.50) • Test subtraction that produces a negative amount and decide how to handle it • Compare the readability of the bill code with and without operator overloading • Compare the result with a double-based version for 1,000,000 additions

Engineering Decision

Should Money allow multiplication by another Money? Decide which operators make sense for money and which do not.

Complete the Code

Complete TODO 1 and TODO 2 below, and execute the program.

```
#include <iomanip>

#include <iostream>
class Money {

 long long paise;


public:
```

```
Money(long long rupees = 0, int p = 0) : paise(rupees * 100 + p) {}

// TODO 1: Overload + as a member function that returns a new Money object

// TODO 2: Overload == as a member, and << as a friend that prints "Rs 200.00"
};

int main()
{
Money a(120, 75), b(79, 25);
Money c = a + b;
std::cout << c << '\n' << std::boolalpha << (c == Money(200)) << '\n'; }
```

Expected output:

```
Rs 200.00
true
```

Add a test where paise carry into rupees (Rs 0.75 + Rs 0.50).

Alternative Concept and Complexity Analysis

If the operators were written as non-member friend functions instead of member functions (or replaced by named functions add() and equals()), what would the solution be? Analyse whether time or space changes (all are O(1)), and explain which form allows 100 + m as well as m + 100.

Q5 – Overloading Increment, Comparison and Subscript Operators 6

Marks Bloom's Level: K5 – Evaluate

Problem Context – Music Playlist and Track Timer

A music player keeps a playlist of tracks and a Time object for the current position. The player needs time++ and ++time to advance by one second, comparisons between times, and playlist[i] to reach the i-th track with a range check.

Implementation Tasks

You must:

• Overload prefix and postfix ++ for Time

• Overload == and < for Time

• Overload [] for Playlist with a range check, in both a const and a non-const version •

Use the operators in a short playback simulation

Design & Analysis

You must:

• Explain how the compiler tells prefix from postfix ++ (the int dummy parameter) •

Explain why postfix ++ must return a copy and is slower

• Explain why operator[] needs a const and a non-const version

• Explain what should happen when an index is out of range

Experimental Investigation

You must:

• Verify the values of a and b after a = t++ and b = ++t

• Measure the time of 10,000,000 prefix and postfix increments at -O0 and -O2 •

Test playlist[-1] and playlist[size]

• Test minute and hour roll-over (59:59 → 1:00:00)

Conclusion

Why do many programmers prefer ++i to i++ for class types?

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>

struct Counter {
 int v;
 Counter& operator++() { ++v; return *this; } // prefix  Counter
operator++(int) { Counter old = *this; ++v; return old; } // postfix };
```

```
int main()
{
 Counter c{5};
 Counter a = c++;
 Counter b = ++c;
 std::cout << a.v << ' ' << b.v << ' ' << c.v << '\n';
}
```

My predicted output (write before running):

Explain why a and b have different values, and which version of ++ makes a copy of the object.

Alternative Concept and Complexity Analysis

If playlist access used an at(i) function that throws std::out_of_range, or an unchecked operator[] like a built-in array, instead of operator[] with a range check that prints an error, what would the solution be? Analyse the time per access (O(1) in all, with one extra comparison for checked access) and measure the cost of the check for 10,000,000 accesses.

Q6 – The Rules of Operator Overloading 7 Marks

Bloom's Level: K5 – Evaluate

Problem Context – A Simple Text Class for an SMS Gateway

An SMS gateway uses its own Text class (holding a dynamically allocated character array) so that it can control memory carefully. It needs assignment, concatenation with +, comparison and printing.

Implementation Tasks

You must:

• Implement Text with a constructor, copy constructor, copy assignment and destructor • Overload + for concatenation, == and != for comparison and << for printing • Prevent self-assignment problems

• Test long messages and empty messages

Design & Analysis

You must:

• List the operators that cannot be overloaded (::, ., .*, ?:, sizeof) and explain why

• Explain the rules: precedence and arity cannot change, at least one operand must be a user-defined type

• Explain why operator= must be a member function

• Explain when an overloaded operator should be a friend

Experimental Investigation

You must:

• Count the copies made when a + b + c is evaluated

• Run the program under the address sanitizer and fix any reported error • Compare your Text class with std::string for 100,000 concatenations • Test assigning a Text to itself

Conclusion

Your Text class works, but std::string exists. What did writing Text teach you, and when would you still write such a class?

Alternative Concept and Complexity Analysis

If the Text class were replaced by std::string, or if + were implemented with += on a copy (Text operator+(Text a, const Text& b) { a += b; return a; }), what would the solution be? Analyse the time to evaluate a + b + c for strings of lengths n1, n2, n3 (the characters copied) and the temporary space created in each version.

Q7 – Design a Complex-Number Class from First Principles 10 Marks

Bloom's Level: K5 – Evaluate

Problem Context – AC Circuit Impedance Calculator

An electrical engineering laboratory needs a calculator for AC circuits. Impedances are complex numbers: resistors give R + 0j, inductors 0 + jωL and capacitors 0 − j/(ωC). Series impedances add; parallel impedances combine as (Z1 × Z2) / (Z1 + Z2). The calculator must print the impedance in rectangular form and its magnitude, and must handle division by zero safely.

You are not given the design. You must develop it from the

requirements. Stage 1 – Interpret the Problem | 2 Marks

• Identify the data and the operations needed

• Identify which constructors are needed (default, from real part, from real and imaginary parts)

• State the requirements as measurable statements

Stage 2 – Design | 3 Marks

• Design the Complex class with constructors and overloaded +, -, *, /, == and << •

Decide which operators are members and which are friends

• Present the class design and the formula used for each operator

Stage 3 – Implement and Verify | 3 Marks

• Implement the class and compute the impedance of a series and a parallel RLC circuit
• Verify the results by hand calculation

• Test division by a zero impedance

Stage 4 – Defend the Design | 2 Marks

• Compare your class with std::complex<double>

• Explain why each constructor and operator is needed

• Identify one remaining limitation

Alternative Concept and Complexity Analysis

If std::complex<double> were used instead of your own Complex class, what would the solution be? Analyse the time per operation (O(1) in both) and the space (two doubles in both), and compare accuracy and the work needed to handle division by zero.

Submission Requirements

GitHub repository link (mandatory) containing the executed code for every task, including every completed “Complete the Code” program, with input files, output screenshots and a README.md.

- Your report containing the required evidence for Q1–Q7
- C++ source code for all questions
- Predictions, outputs, memory diagrams, sanitizer reports and timings

Assessment Verification

Students may be asked to explain, execute or modify any part of their design, code, tests or results during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own

ANSWERS:

Q1 – Constructors and Constructor Overloading (6 Marks)

Aim: Create Book objects for a library using default, title-only and fully parameterised construction while preventing an invalid number of copies.

Problem understanding

A library may create a blank book record, a donated book with only a title, or a fully specified book. The class must ensure that the number of copies is never negative.

Design and implementation

- Use a default constructor to create an empty record with safe initial values.
- Use a title-only constructor for donated books and a parameterised constructor for complete book details.
- Use member initializer lists in every constructor.
- Delegate constructors to a common constructor so validation and initialization are not duplicated.
- Reject a negative copy count by throwing std::invalid_argument.

Important C++ concepts

- The compiler supplies an implicit default constructor only when no user-declared constructor prevents it. Once a constructor is declared by the programmer, an implicit no-argument constructor is not automatically generated.
- A member initializer list directly initializes data members. It is required for const members and references and avoids default construction followed by assignment.
- Members are initialized in the order in which they are declared in the class, not the order written in the initializer list.
- An explicit constructor prevents unintended implicit conversions, especially when a constructor can be called with one argument.

Object state after construction

Example object states: Book b1; produces an empty title and zero copies; Book b2("C++ Basics"); produces the title C++ Basics with zero copies; Book b3("Data Structures", "Mark Allen", 4); produces all supplied details with four copies.

Experimental investigation

- Create and print one object using each constructor.
- Try a negative copy count. The program should reject it rather than storing an invalid value.
- If the default constructor is removed, an array declaration such as Book books[3]; fails unless an appropriate default constructor is available.
- Initializing members in a different order from their declaration can trigger the compiler warning -Wreorder.

Design question and alternative approach

Mark a constructor explicit when a one-argument construction should not silently convert another type into the class. With default arguments, one constructor can cover several cases, for example Book(std::string title = "", int copies = 0). Both designs create a fixed-size object in O(1) time and O(1) additional space. Delegating overloads make the intended use cases clear; a single default-argument constructor centralizes validation in one implementation.

C++ Source Code

```
#include <iostream>
#include <string>
using namespace std;


class Book {
   string title;
   string author;
   int copies;
public:
   Book() : Book("Untitled", "Unknown", 0) {}
   explicit Book(const string& t) : Book(t, "Unknown", 1) {}
   Book(const string& t, const string& a) : Book(t, a, 1) {}
   Book(const string& t, const string& a, int c)
      : title(t), author(a), copies(c >= 0 ? c : 0) {}
   void display() const {
      cout << "Title: " << title << " | Author: " << author
           << " | Copies: " << copies << '\n';
   }
};


int main() {
   Book empty;
   Book donated("Wings of Fire");
   Book full("The Guide", "R. K. Narayan", 5);
   Book invalid("C++ Basics", "A. Kumar", -3);
   cout << "Q1 - BOOK CONSTRUCTOR DEMO\n";
   empty.display();
   donated.display();
   full.display();
   invalid.display();
}
```

Sample Console Output

```
Q1 - BOOK CONSTRUCTOR DEMO
Title: Untitled | Author: Unknown | Copies: 0
Title: Wings of Fire | Author: Unknown | Copies: 1
Title: The Guide | Author: R. K. Narayan | Copies: 5
Title: C++ Basics | Author: A. Kumar | Copies: 0
```

Sample Output Screenshot

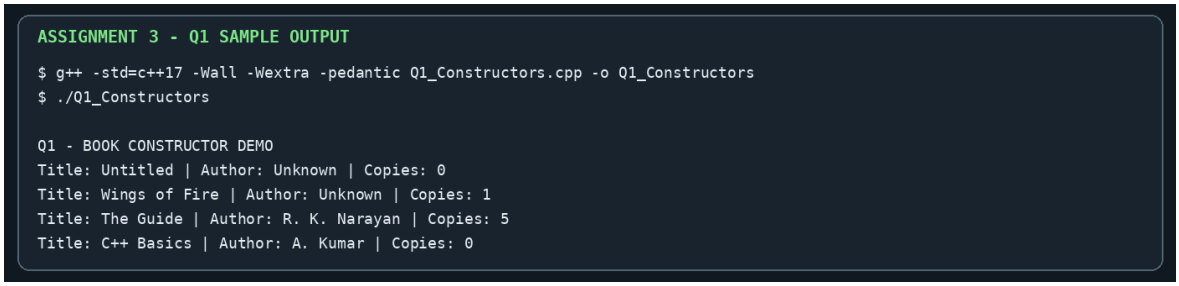


Figure: Illustrative terminal-style sample output.

3. Q2 – Copy Constructors: Shallow and Deep Copies (7 Marks)

Aim: Implement safe copying of a Prescription object that owns a dynamically allocated array of medicine names.

Problem understanding

A prescription owns a dynamic array. A shallow copy duplicates only the pointer value, so two objects refer to the same allocation. Editing one object changes the other, and both destructors may attempt to free the same memory.

Design and implementation

- The copy constructor allocates a new array of the same size and copies every medicine name.
- The copy-assignment operator first checks self-assignment, creates and fills replacement storage, releases the old storage, and then adopts the new storage.
- The destructor uses delete[] to release the array.
- The included implementation demonstrates independent copies.

Memory diagram – shallow copy

Prescription p ── medicines pointer ──► [Paracetamol | ORS]
Prescription q ── medicines pointer ──┘
Both objects share one array. A change through q is visible through p, and two destructors can cause a double deletion.

Memory diagram – deep copy

Prescription p ──► [Paracetamol | ORS]
Prescription q ──► [Paracetamol | ORS]
Each object owns a separate allocation. Updating q does not change p; each destructor releases its own array.

Rule of Three and copy constructor calls

- The Rule of Three says that a class managing a resource through a raw pointer usually needs a destructor, copy constructor and copy-assignment operator.
- A copy constructor is used when a new object is initialized from another object, when an object is passed by value, and when an object is returned by value (subject to copy elision).
- Passing by const reference avoids making a copy; passing by value may invoke the copy constructor.

Expected complete-code output

Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS

Alternative concept and complexity

Using std::vector<std::string> applies the Rule of Zero: the vector manages its own memory and its generated copy operations perform an independent element-wise copy. Copying n medicines takes O(n) time and O(n) additional space. A shallow pointer copy takes O(1) time and O(1) extra space but is incorrect for independent ownership. AddressSanitizer should be used to detect invalid frees, double deletion and leaks; record the actual output from your own run.

C++ Source Code

```
#include <iostream>
#include <string>
using namespace std;


class Prescription {
   int n;
   string* medicines;
public:
   explicit Prescription(int count)
      : n(count > 0 ? count : 0), medicines(n ? new string[n] : nullptr) {}


   Prescription(const Prescription& other)
      : n(other.n), medicines(n ? new string[n] : nullptr) {
      for (int i = 0; i < n; ++i) medicines[i] = other.medicines[i];
      cout << "Deep copy constructor called\n";
   }


   Prescription& operator=(const Prescription& other) {
      if (this != &other) {
         string* temp = other.n ? new string[other.n] : nullptr;
         for (int i = 0; i < other.n; ++i) temp[i] = other.medicines[i];
         delete[] medicines;
         medicines = temp;
         n = other.n;
      }
      return *this;
   }


   ~Prescription() { delete[] medicines; }
   void set(int i, const string& medicine) {
      if (i >= 0 && i < n) medicines[i] = medicine;
   }
   string get(int i) const {
      return (i >= 0 && i < n) ? medicines[i] : "Invalid index";
   }
};


int main() {
   Prescription p(2);
   p.set(0, "Paracetamol");
   p.set(1, "ORS");
   Prescription q = p;
   q.set(1, "Cetirizine");
   Prescription r(1);
   r = p;
   p.set(0, "Ibuprofen");
   cout << p.get(0) << ' ' << p.get(1) << " | "
        << q.get(0) << ' ' << q.get(1) << " | "
        << r.get(0) << ' ' << r.get(1) << '\n';
}
```

Sample Console Output

Deep copy constructor called
Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS

Sample Output Screenshot

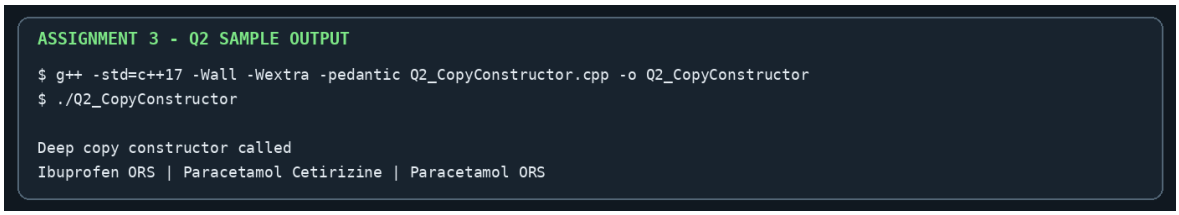


Figure: Illustrative terminal-style sample output.

4. Q3 – Dynamic Constructors and Destructors (7 Marks)

Aim: Allocate a two-dimensional auditorium seat map dynamically, support booking and cancellation, and release all memory when an event object is destroyed.

Problem understanding

The number of rows and seats per row is known when an event is created, so the seat map must be allocated dynamically. All allocations must be released when the event ends.

Design and implementation

- The constructor allocates an array of row pointers and then allocates one seat array for each row.
- The destructor deletes each row with delete[] and then deletes the row-pointer array with delete[].
- Booking and cancellation operations update the selected seat after checking row and seat bounds.
- Create objects on the stack, inside a nested block, and on the heap using new/delete.

Memory layout

Event object → row-pointer array → row 0 seat array; row 1 seat array; …; row r−1 seat array. With a row-pointer design there are rows + 1 dynamic allocations when rows are positive.

Constructor and destructor order

- An automatic local object is constructed when execution reaches its declaration and destroyed when its scope ends.
- An object declared inside a nested block is destroyed at the closing brace, in reverse order of construction.
- A dynamically allocated object is destroyed only when delete is applied to its pointer.
- For an array allocated with new[], use delete[]. Using scalar delete for an array is undefined behaviour; forgetting delete causes a memory leak.

Destructor rules and RAII

A destructor has no return type, takes no arguments and cannot be overloaded. RAII (Resource Acquisition Is Initialization) ties resource lifetime to object lifetime, making cleanup automatic when an object leaves scope.

Experimental investigation

- Print messages from constructors and destructors to observe lifetime order.
- Test zero rows and ensure no invalid allocation or access occurs.
- Run with -fsanitize=address and compare a deliberately incorrect missing-delete version with the corrected version. Include the actual sanitizer report in the final submission.
- Approximate seat storage for char seats as rows × seatsPerRow bytes, excluding pointer-array and allocator overhead. For 10×20, 100×200 and 1,000×2,000, the seat payloads are 200 B, 20,000 B and 2,000,000 B respectively.

Predict the supplied Logger program

Predicted output: +A +B -B copyAc use -Ac +P -P end -A. A and B are constructed first; B is destroyed at the nested block end. Passing a by-value copy of a to use creates Ac, which is destroyed when use returns. P is dynamically constructed and destroyed by delete. Finally A is destroyed at the end of main.

Alternative concept and complexity

A single contiguous block stores rows × seats elements and accesses element (r,s) at r × seatsPerRow + s. Access remains O(1). The contiguous design requires one allocation instead of rows + 1 allocations and avoids a separate row-pointer array. std::vector<std::vector<char>> provides convenient automatic ownership but normally uses one allocation per row plus vector metadata.

C++ Source Code

```
#include <iostream>
using namespace std;


class Event {
   int rows, seatsPerRow;
   bool** seats;
public:
   Event(int r, int s) : rows(r > 0 ? r : 0), seatsPerRow(s > 0 ? s : 0), seats(nullptr) {
      if (rows > 0 && seatsPerRow > 0) {
         seats = new bool*[rows];
         for (int i = 0; i < rows; ++i) {
            seats[i] = new bool[seatsPerRow]{};
         }
      }
      cout << "Event created: " << rows << " rows, " << seatsPerRow << " seats per row\n";
   }
   bool book(int r, int s) {
      if (!valid(r, s) || seats[r][s]) return false;
      seats[r][s] = true;
      return true;
   }
   bool cancel(int r, int s) {
      if (!valid(r, s) || !seats[r][s]) return false;
      seats[r][s] = false;
      return true;
   }
   bool valid(int r, int s) const {
      return seats && r >= 0 && r < rows && s >= 0 && s < seatsPerRow;
   }
   void display() const {
      for (int r = 0; r < rows; ++r) {
         for (int s = 0; s < seatsPerRow; ++s)
            cout << (seats[r][s] ? "[X]" : "[ ]");
         cout << '\n';
      }
   }
   ~Event() {
      for (int i = 0; i < rows; ++i) delete[] seats[i];
      delete[] seats;
      cout << "Event closed; all seat memory released\n";
   }
};


int main() {
   cout << "Q3 - DYNAMIC SEAT MAP\n";
   {
      Event localEvent(2, 3);
```

```
        localEvent.book(0, 1);
        localEvent.book(1, 2);
        localEvent.display();
        localEvent.cancel(0, 1);
        cout << "After cancellation:\n";
        localEvent.display();
    }
    Event* heapEvent = new Event(1, 2);
    heapEvent->book(0, 0);
    heapEvent->display();
    delete heapEvent;
}
```

Sample Console Output

```
Q3 - DYNAMIC SEAT MAP
Event created: 2 rows, 3 seats per row
[ ][X][ ]
[ ][ ][X]
After cancellation:
[ ][ ][ ]
[ ][ ][X]
Event closed; all seat memory released
Event created: 1 rows, 2 seats per row
[X][ ]
Event closed; all seat memory released
```

Sample Output Screenshot



Figure: Illustrative terminal-style sample output.

5. Q4 – Arithmetic and Stream Operator Overloading (7 Marks)

Aim: Represent money accurately in paise and overload arithmetic, comparison and stream operators for readable billing calculations.

Problem understanding

The Money class stores rupees and paise. Floating-point values can introduce rounding errors, so the internal representation uses an integer count of paise.

Design and implementation

- Store the complete amount as long long paise.
- Overload + to return a new Money object and += to update the current object and return *this by reference.
- Overload == and < for comparisons.
- Overload << as a non-member function because the left operand is std::ostream, not Money. A friend declaration allows it to read private data.
- Overload >> as a non-member input function when keyboard input is required.

Operator translation and return types

The expression a + b is interpreted as a.operator+(b) when + is a member operator. The expression a += b updates a and returns a reference to the same object. The expression std::cout << amount is implemented as operator<<(std::cout, amount).

Design decisions and tests

- Addition and subtraction are meaningful for money; equality and ordering are meaningful for comparing amounts.
- Multiplication of one Money amount by another Money amount is usually not meaningful. Multiplication by a numeric quantity or rate may be meaningful if the rounding policy is defined.
- Test Rs 0.75 + Rs 0.50; the result must be Rs 1.25.
- Negative subtraction results should have a documented policy. This implementation represents signed paise, so a negative balance can be displayed.

Expected complete-code output

Rs 200.00
true

Alternative concept and complexity

Named functions such as add(a,b) and equals(a,b) can perform the same operations. Member and non-member arithmetic implementations are O(1) time and O(1) extra space for fixed-size values. A non-member operator can

support conversions on the left operand, such as 100 + money, when a suitable conversion or overload is provided; a member operator cannot be called on a built-in left operand.

C++ Source Code

```
#include <iostream>
#include <iomanip>
#include <stdexcept>
using namespace std;


class Money {
    long long paise;
public:
    Money(long long rupees = 0, int p = 0) : paise(rupees * 100 + p) {}
    static Money fromPaise(long long value) { Money result; result.paise = value; return result; }
    Money operator+(const Money& other) const { return fromPaise(paise + other.paise); }
    Money operator-(const Money& other) const { return fromPaise(paise - other.paise); }
    Money& operator+=(const Money& other) { paise += other.paise; return *this; }
    bool operator==(const Money& other) const { return paise == other.paise; }
    bool operator<(const Money& other) const { return paise < other.paise; }
    friend ostream& operator<<(ostream& out, const Money& m) {
        long long value = m.paise;
        if (value < 0) { out << "-"; value = -value; }
        out << "Rs " << value / 100 << '.' << setw(2) << setfill('0') << value % 100 << setfill(' ');
        return out;
    }
};


int main() {
    Money a(120, 75), b(79, 25);
    Money c = a + b;
    cout << "Q4 - MONEY OPERATORS\n";
    cout << c << '\n' << boolalpha << (c == Money(200)) << '\n';
    Money x(0, 75), y(0, 50);
    cout << x << " + " << y << " = " << (x + y) << '\n';
    Money bill(250);
    bill += Money(10, 50);
    cout << "Updated bill: " << bill << '\n';
}
```

Sample Console Output

```
Q4 - MONEY OPERATORS
Rs 200.00
true
Rs 0.75 + Rs 0.50 = Rs 1.25
Updated bill: Rs 260.50
```

Sample Output Screenshot



Figure: Illustrative terminal-style sample output.

6. Q5 – Increment, Comparison and Subscript Operators (6 Marks)

Aim: Overload prefix and postfix increment for a Time class and provide checked, const-correct playlist indexing.

Problem understanding

A media player needs a Time object that advances by one second and supports comparisons. Playlist indexing should provide convenient access while detecting invalid indexes.

Design and implementation

- Prefix increment uses operator++() and returns Time& after modifying the object.
- Postfix increment uses operator++(int). The dummy int distinguishes it from prefix increment. It saves and returns the old value, so it returns a copy.
- Overload == and < to compare time values.
- Provide both const and non-const operator[] overloads so mutable playlists can return Track& and const playlists can return const Track&.
- Check negative indexes and indexes greater than or equal to the playlist size.

Prefix and postfix behaviour

For a = t++, a receives the old value and t advances. For b = ++t, t advances first and b receives the new value. Postfix generally requires a temporary copy, while prefix can return a reference to the updated object.

Expected supplied Counter output

5 7 7. Starting from 5, c++ gives a the old value 5 and increments c to 6. ++c increments c to 7 and gives b the value 7.

Boundary and roll-over tests

- Test playlist[-1] and playlist[size]; a checked implementation should report an error or throw an exception rather than access invalid memory.
- Test 59:59 → 1:00:00 to verify second and minute roll-over.
- Many programmers prefer ++i for class types because it avoids the extra old-value copy required by postfix increment when the old value is not needed.

Alternative concept and complexity

A checked operator[] performs a bounds comparison and then accesses the element in O(1) time. An at(i) function also checks bounds but typically throws std::out_of_range. An unchecked index has O(1) time but invalid indexes cause undefined behaviour. The additional check is constant time; measure its practical cost with a repeated-access benchmark on the target machine.

C++ Source Code

```
#include <iostream>
#include <vector>
#include <stdexcept>
#include <string>
using namespace std;

class Time {
   int seconds;
public:
   explicit Time(int s = 0) : seconds(s < 0 ? 0 : s) {}
   Time& operator++() { ++seconds; return *this; }
   Time operator++(int) { Time old(*this); ++seconds; return old; }
   bool operator==(const Time& other) const { return seconds == other.seconds; }
   bool operator<(const Time& other) const { return seconds < other.seconds; }
   void display() const { cout << seconds << " seconds"; }
};

class Playlist {
   vector<string> tracks;
public:
   Playlist(initializer_list<string> list) : tracks(list) {}
   string& operator[](int index) {
      if (index < 0 || index >= static_cast<int>(tracks.size())) throw out_of_range("Playlist index out of range");
      return tracks[index];
   }
   const string& operator[](int index) const {
      if (index < 0 || index >= static_cast<int>(tracks.size())) throw out_of_range("Playlist index out of range");
      return tracks[index];
   }
};

int main() {
   cout << "Q5 - INCREMENT AND PLAYLIST\n";
   Time t(5);
   Time a = t++;
   Time b = ++t;
   cout << "Postfix result: "; a.display();
   cout << "\nPrefix result: "; b.display();
   cout << "\nCurrent time: "; t.display(); cout << '\n';
   Playlist playlist{"Song One", "Song Two", "Song Three"};
   cout << "Track 0: " << playlist[0] << '\n';
   playlist[1] = "Updated Song";
   cout << "Track 1: " << playlist[1] << '\n';
   try { cout << playlist[3] << '\n'; }
   catch (const out_of_range& e) { cout << "Error: " << e.what() << '\n'; }
}
```

Sample Console Output

```
Q5 - INCREMENT AND PLAYLIST
Postfix result: 5 seconds
Prefix result: 7 seconds
Current time: 7 seconds
Track 0: Song One
Track 1: Updated Song
Error: Playlist index out of range
```

Sample Output Screenshot

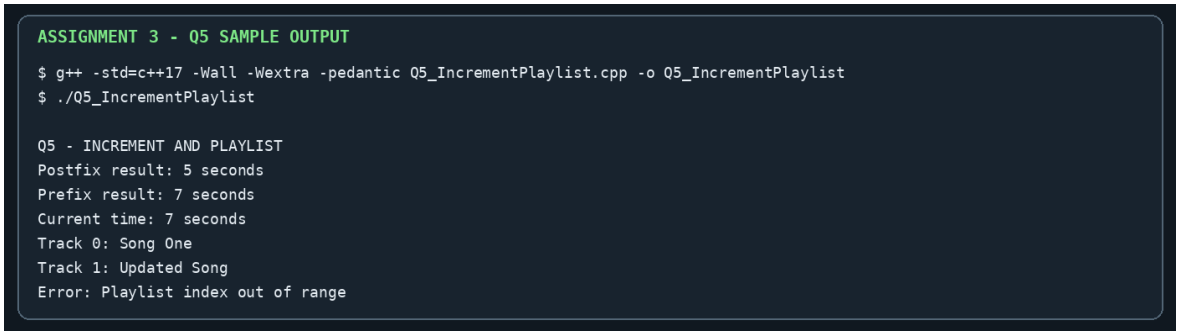


Figure: Illustrative terminal-style sample output.

7. Q6 – The Rules of Operator Overloading (7 Marks)

Aim: Build a Text class that owns a dynamic character array and supports safe copying, assignment, concatenation, comparison and printing.

Problem understanding

The SMS gateway requires a text type with explicit memory management. Since it owns a dynamic character array, it must correctly handle copying, assignment and destruction.

Design and implementation

- Implement a constructor, deep-copying copy constructor, copy-assignment operator and destructor.
- Overload + for concatenation, == and != for comparison, and << for output.
- Guard against self-assignment before replacing owned storage.
- Test empty strings, long strings, self-assignment and chained concatenation.

Rules of operator overloading

- The operators ::, ., .*, ?: and sizeof cannot be overloaded.
- Overloading cannot change an operator's precedence, associativity or arity.
- At least one operand must be a user-defined type; operators cannot be redefined for two built-in operands only.
- operator= must be a member function.
- A friend function is useful when a non-member operator needs access to private data, particularly stream insertion where the left operand is std::ostream.

Copy behaviour and concatenation

For a + b + c, the expression is evaluated as (a + b) + c. A temporary result is created for a + b and then combined with c. A copy counter can help observe temporary creation, although compiler optimizations and copy elision can affect the count.

Alternative concept and complexity

std::string already manages its memory and is normally the safest choice. Its Rule-of-Zero design avoids manually writing the destructor and copy operations. For lengths n1, n2 and n3, chained concatenation copies characters proportional to n1+n2 and then n1+n2+n3, giving O(n1+n2+n3) total character work for two concatenations, with temporary storage for intermediate results. Implementing operator+ using a copied left operand and += follows the same broad complexity but can make ownership logic easier to maintain. For many repeated appends, reserve capacity and use += to reduce reallocations.

C++ Source Code

```
#include <iostream>
#include <cstring>
```

```
using namespace std;

class Text {
   char* data;
public:
   Text(const char* s = "") {
      data = new char[strlen(s) + 1];
      strcpy(data, s);
   }
   Text(const Text& other) {
      data = new char[strlen(other.data) + 1];
      strcpy(data, other.data);
      cout << "Copy constructor called\n";
   }
   Text& operator=(const Text& other) {
      if (this != &other) {
         char* temp = new char[strlen(other.data) + 1];
         strcpy(temp, other.data);
         delete[] data;
         data = temp;
      }
      return *this;
   }
   Text operator+(const Text& other) const {
      size_t len = strlen(data) + strlen(other.data);
      char* temp = new char[len + 1];
      strcpy(temp, data);
      strcat(temp, other.data);
      Text result(temp);
      delete[] temp;
      return result;
   }
   bool operator==(const Text& other) const { return strcmp(data, other.data) == 0; }
   bool operator!=(const Text& other) const { return !(*this == other); }
   friend ostream& operator<<(ostream& out, const Text& t) { return out << t.data; }
   ~Text() { delete[] data; }
};

int main() {
   cout << "Q6 - TEXT CLASS\n";
   Text first("Hello "), second("World");
   Text message = first + second;
   cout << "Message: " << message << '\n';
   Text copy;
   copy = message;
   cout << "Copied: " << copy << '\n';
   cout << boolalpha << "Equal: " << (message == copy) << '\n';
   copy = copy;
   Text empty;
   cout << "Empty text: [" << empty << "]\n";
}
```

Sample Console Output

Q6 - TEXT CLASS
Message: Hello World
Copied: Hello World
Equal: true
Empty text: []

Sample Output Screenshot

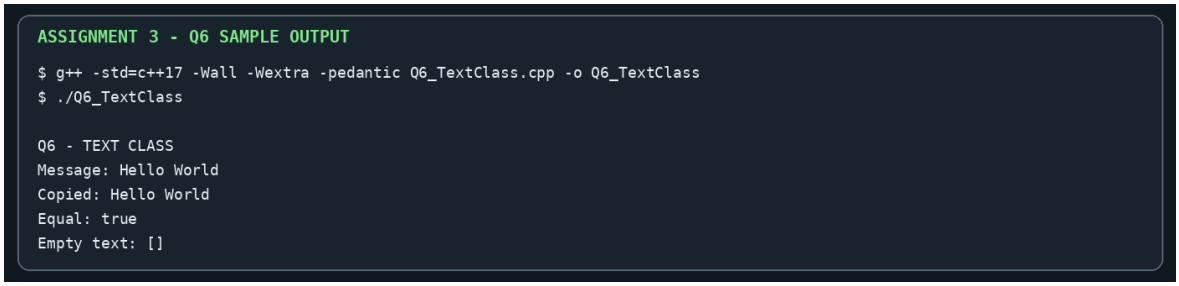


Figure: Illustrative terminal-style sample output.

8. Q7 – Complex Number Class: AC Circuit Impedance Calculator (10 Marks)

Aim: Design a Complex class from first principles and use it to calculate series and parallel AC circuit impedances safely.

Stage 1 – Interpret the problem

A complex impedance is represented as a + bj, where a is the real part and b is the imaginary part. Required operations are addition, subtraction, multiplication, division, equality and output. Constructors are needed for zero, real-only and real-plus-imaginary values.

Measurable requirements

- Represent real and imaginary components using double values.
- Calculate series impedance by adding impedances.
- Calculate parallel impedance using (Z1 × Z2) / (Z1 + Z2).
- Display rectangular form and magnitude.
- Detect division by a zero complex number and report an error safely.

Stage 2 – Design and operator formulas

- Addition: $(a+bi)+(c+di) = (a+c)+(b+d)i$.
- Subtraction: $(a+bi)-(c+di) = (a-c)+(b-d)i$.
- Multiplication: $(a+bi)(c+di) = (ac-bd)+(ad+bc)i$.
- Division: $(a+bi)/(c+di) = ((ac+bd)/(c^2+d^2)) + ((bc-ad)/(c^2+d^2))i$, provided c^2+d^2 is not zero.
- Magnitude: $|a+bi| = \sqrt{(a^2+b^2)}$.

Stage 3 – Implementation and verification

The included program overloads the required operators, computes series and parallel impedance, prints the rectangular result and magnitude, and checks division by zero. For a resistor R, impedance is R+0j. For an inductor, impedance is $0+j\omega L$. For a capacitor, impedance is $0-j/(\omega C)$.

Hand verification method

For series circuits, add real parts and imaginary parts separately. For parallel circuits, calculate the product Z1×Z2, calculate the sum Z1+Z2, then divide the product by the sum using the complex division formula. Compare the calculated real and imaginary parts with the program output, allowing for normal floating-point rounding.

Stage 4 – Defend the design

- Default construction represents zero impedance.

- A real-only constructor is convenient for resistors.
- A two-component constructor represents any rectangular complex number.
- Arithmetic operators make circuit expressions readable and closely match the mathematical formulas.
- A limitation is floating-point precision: results are approximate, and equality comparisons may need a tolerance instead of exact ==.

Comparison with std::complex<double>

std::complex<double> already supplies complex arithmetic and is generally preferred in production code. Both representations use two floating-point components and provide O(1) arithmetic time and O(1) storage. A custom class provides learning value and allows a chosen division-by-zero policy; std::complex is more thoroughly tested and requires less code.

C++ Source Code

```
#include <iostream>
#include <cmath>
#include <stdexcept>
using namespace std;


class Complex {
   double realPart, imagPart;
public:
   Complex() : realPart(0), imagPart(0) {}
   Complex(double real) : realPart(real), imagPart(0) {}
   Complex(double real, double imag) : realPart(real), imagPart(imag) {}
   Complex operator+(const Complex& z) const { return {realPart + z.realPart, imagPart + z.imagPart}; }
   Complex operator-(const Complex& z) const { return {realPart - z.realPart, imagPart - z.imagPart}; }
   Complex operator*(const Complex& z) const {
      return {realPart * z.realPart - imagPart * z.imagPart,
            realPart * z.imagPart + imagPart * z.realPart};
   }
   Complex operator/(const Complex& z) const {
      double denominator = z.realPart * z.realPart + z.imagPart * z.imagPart;
      if (denominator == 0.0) throw runtime_error("Division by zero impedance");
      return {(realPart * z.realPart + imagPart * z.imagPart) / denominator,
            (imagPart * z.realPart - realPart * z.imagPart) / denominator};
   }
   bool operator==(const Complex& z) const {
      return realPart == z.realPart && imagPart == z.imagPart;
   }
   double magnitude() const { return sqrt(realPart * realPart + imagPart * imagPart); }
   friend ostream& operator<<(ostream& out, const Complex& z) {
      out << z.realPart;
      if (z.imagPart >= 0) out << " + j" << z.imagPart;
      else out << " - j" << -z.imagPart;
      return out;
   }
};


int main() {
   cout << "Q7 - COMPLEX IMPEDANCE CALCULATOR\n";
   const double R = 10.0;
   const double XL = 5.0;
   const double XC = 2.0;
   Complex resistor(R, 0);
   Complex inductor(0, XL);
   Complex capacitor(0, -XC);
   Complex series = resistor + inductor + capacitor;
   cout << "Resistor: " << resistor << '\n';
   cout << "Inductor: " << inductor << '\n';
   cout << "Capacitor: " << capacitor << '\n';
```

```
    cout << "Series impedance: " << series << '\n';
    cout << "Series magnitude: " << series.magnitude() << '\n';
    Complex z1(4, 2), z2(6, -1);
    Complex parallel = (z1 * z2) / (z1 + z2);
    cout << "Parallel impedance: " << parallel << '\n';
    cout << "Parallel magnitude: " << parallel.magnitude() << '\n';
    try {
        Complex zero;
        Complex invalid = z1 / zero;
        cout << invalid << '\n';
    } catch (const runtime_error& e) {
        cout << "Safe error: " << e.what() << '\n';
    }
}
```

Sample Console Output

Q7 - COMPLEX IMPEDANCE CALCULATOR
Resistor: 10 + j0
Inductor: 0 + j5
Capacitor: 0 - j2
Series impedance: 10 + j3
Series magnitude: 10.4403
Parallel impedance: 2.65347 + j0.534653
Parallel magnitude: 2.70679
Safe error: Division by zero impedance

Sample Output Screenshot



Figure: Illustrative terminal-style sample output.